Zero → Production Multi-Agent AI Engineer — Curriculum

Mentor-mode reference document. Lives in `multi-agent-ai-engineering-curriculum/`, updated as we go. Rules we're following (from you, kept short):

- You write all the code. I give requirements, hints, reviews, bugs, and interview questions — not solutions, unless you say **"Show me the solution."**
- Every topic gets: WHAT / WHY / WHEN / WHERE / HOW / HOW IT WORKS UNDERNEATH / ALTERNATIVES / TRADE-OFFS / COMMON MISTAKES / HOW IT'S USED IN PRODUCTION.
- Every topic goes: theory → practice (you code) → build → break → debug → explain → interview practice.
- No document starts until you type **START DOCUMENT 01** (or the number of whichever document you want).
- Folder rule: each document gets a `NN_topic_name/` folder, and each project gets a `project_N_name/` folder, made when we actually start it — not built ahead of time.

Jump to: 1. Order · 2. Document order · 3. Today's tools · 4. Document breakdown · 5. Multi-agent patterns · 6. Final capabilities · Status

1. Order, and where I changed yours

Your order was already good. Three small changes, with reasons:

Change	Reason
Async Python is now a **required checkpoint before Document 09 (LangGraph)**, not its own numbered document	LangGraph's way of running things (streaming, running steps at the same time, `ainvoke`) assumes you're comfortable with `async` / `await`. Teaching it right before Doc09, instead of bolting it onto the Python basics document (where it has no real use case yet), avoids two problems at once.
RAG (Doc08) comes before LangGraph (Doc09), and Project 3 uses RAG as a search tool inside the LangGraph agent	Your project list didn't have a stand-alone "RAG project" — RAG becomes one tool a LangGraph agent can use. Building the search piece as plain Python first (Doc08) means Doc09 only has to teach graph mechanics, not graph mechanics + embeddings at the same time.

Change	Reason
SQL/Databases and Docker aren't their own documents — they're folded into **Document 12 (Production Engineering)** as their own sections	You listed them in the tech stack, but not in the 18-document list. They only matter once there's a real system worth saving data for and shipping, which is exactly Doc12's job. Splitting them out would repeat the "production" framing three separate times.

Everything else matches your numbering exactly.

---

2. Order of documents

```
01 Python & Engineering Foundations
        ↓
02 APIs, HTTP, JSON & Backend Basics
        ↓
03 LLM Basics
        ↓
04 OpenAI API  ──────────────► PROJECT 1 (Beginner LLM App)
        ↓
05 LangChain Basics
        ↓
06 Tools & Function Calling
        ↓
07 AI Agents  ──────────────► PROJECT 2 (Tool-Using Agent)
        ↓
08 RAG
        ↓
   [required checkpoint: Async Python]
        ↓
09 LangGraph
        ↓
10 Agent Workflows  ─────────► PROJECT 3 (LangGraph App + RAG tool)
        ↓
11 Multi-Agent Systems  ─────► PROJECT 4 (Supervisor + 3-5 agents)
        ↓
12 Production AI Engineering (FastAPI, database, Docker, security, cost)
        ↓
13 Testing, Evaluation & Watching Your System
        ↓
14 Debugging Lab (every failure type from Docs 01-13, practiced together)
        ↓
15 Five Projects (an index linking each project brief to its documents)
        ↓
16 System Design & Architecture
```

```
        ↓
17 Interview Preparation (pulls interview practice from every document)
        ↓
18 Final Production Multi-Agent Capstone  ⟶ PROJECT 5 (Production system)
        ↓
19 MLOps & LLMOps (versions, gated releases, rollback, slow-decline watch)
```

Document 19 got added after your original request, at your ask ("MLOps and Docker stuff too"). It sits after the capstone on purpose — MLOps only makes sense once there's a real system (Project 5) to run. Docker itself is covered where it belongs, inside Doc12, since it's part of shipping *one* version. Doc19 is about safely changing that shipped system afterward.

Reading Doc15 and Doc17 correctly: they're not new content. Doc15 is the one place all five project briefs are indexed (you'll come back to it, not read it top to bottom). Doc17 pulls together and practices the interview questions that already showed up at the end of every document, plus adds system-design-level questions once you know enough to actually answer them.

Project ↔ Document map:

Every project is a real multi-agent system — the number of agents grows 2→2→3→5→5 across the arc, so each project adds one new design idea (routing, then checking, then graph-native handoffs, then dynamic supervising, then production-hardening) instead of several at once. Full step-by-step build guides for each project (with sub-steps, reading links, and growing file structure) live in each project's own README.

Project	Agents	Built during	What it proves
1 — SupportDesk (Concierge & Triage)	2	Docs 01-04	You can talk to a model correctly and safely, and route between two simple agents
2 — ResearchHand (Worker & Verifier)	2	Docs 05-07	You can give a model real abilities, check its choices, and add an independent check
3 — DocuMind (Retriever, Reasoner & Approval)	3	Docs 08-10	You can build agents as a clear, checkable graph, with real handoffs
4 — ContentForge (Supervisor-Led Team)	5	Doc 11	You can split up a problem across specialist agents, with a supervisor deciding routing on the fly
5 — ContentForge Pro (Production Platform)	5 (production-ready)	Docs 12-19	You can ship, test, secure, watch, version, and defend the design of #4
6 (bonus) — LangChainPro	—	Doc05	You can build a real multi-step pipeline (branching, parallel calls, retries/fallbacks)

Project	Agents	Built during	What it proves
(Production LCEL Patterns)			in LangChain alone, without needing LangGraph
7 (bonus) — MCPForge (MCP Server)	— (server, not agents)	Doc06's MCP topics	You can expose real tools/resources/prompts over the Model Context Protocol, usable by any MCP client
8 (bonus) — MCPBridge (MCP Client Agent)	1 (dynamic tools)	Doc06/07's MCP + ReAct topics	You can build an agent whose tools are discovered at runtime from live MCP servers, not hardcoded
9 (bonus) — PromptShield (Injection-Hardened RAG Agent)	1	Doc06/08's prompt injection topics	You can defend a RAG agent against injected instructions, and prove it with a real red-team test suite
10 (bonus) — MemoryKeeper (Persistent Cross-Session Memory)	1	Doc09's long-term memory topic	You can give an agent real memory that survives across separate conversations, with a working "forget me"
11 (bonus) — MCPCrew (Multi-Agent Research Team)	3 (+ Supervisor)	Doc06/11's MCP + multi-agent topics	You can build a Supervisor-led team where each specialist connects to its own MCP server, with graceful degradation
12 (bonus) — MCPResearch (Agentic Pipeline as an MCP Tool)	2 (Researcher + Fact-Checker)	Doc06/11's MCP + multi-agent topics	You can wrap a whole agentic pipeline behind one MCP tool call — the "agent-as-a-service" pattern
13 (bonus) — CodeGuard (PR-Review Team)	4	All of Docs 01-19, applied again	You can apply the whole architecture to a new problem, with less help

3. Notes on today's tools (flagged now, taught in full when we get there)

- **OpenAI's API:** the Chat Completions API is the steady foundation (still what LangChain's `langchain-openai` wraps underneath, and what most existing production code and tutorials use) — we start there in Doc04. OpenAI's newer **Responses API** is today's recommendation for new agent-style, tool-heavy apps (it keeps state built in, and is friendlier for tool use). We'll cover both, and I'll tell you which one a given piece of code is using and why, so you never stare at code that "looks wrong" just because it's a different generation of the API.

- **LangChain agents:** `create_agent` is the current standard way to get a ready-made tool-calling agent; its older predecessor `AgentExecutor` is still common in real codebases and interview questions, worth recognizing. New work involving a graph uses **LangGraph** directly (`create_agent`, or your own `StateGraph`). Doc07 teaches both `create_agent` and the older `AgentExecutor` so you recognize either; Doc09 onward is where you build for real.
- **Multi-agent handoffs:** today's LangGraph pattern is the `Command` tool for handing work between agents and sharing/splitting state, replacing older, hand-made routing-text conventions. Taught in Doc11.
- **Structured output:** Pydantic v2 plus OpenAI's own structured-output/tool-shape support is today's standard. We won't teach the older "manually parse JSON and hope" pattern, except as a "here's why we stopped doing this" example.

---

4. Document-by-document breakdown

Each row below answers, for that document: what topics, what to learn, what to practice, what to build, what to break/debug, what to explain and be interview-ready on, and what has to be true before you move on.

Jump straight to: 01 · 02 · 03 · 04 · 05 · 06 · 07 · 08 · 09 · 10 · 11 · 12 · 13 · 14 · 15 · 16 · 17 · 18 · 19

Document 01 — Python & Engineering Foundations

- **Depends on:** nothing. **Required checkpoint:** none.
- **Topics:** functions, classes, modules/packages, type hints (the `typing` module), errors, virtual environments, pip/managing packages, `.env` + environment variables, logging, basic project layout, Git basics, JSON.
- **Learn:** why each of these exists as an engineering habit, not just syntax — like why secrets never live in code, why a venv exists, why type hints on functions matter once your code is over 200 lines.
- **Practice:** small scripts, one idea at a time, before combining them.
- **Build:** a config-loading module (`config.py`) that reads `.env` safely, checks required keys are there, and fails loudly (not quietly) if one is missing.
- **Break & debug:** a missing environment variable, the wrong type passed to a typed function, import errors from a badly organized package, a `.gitignore` that leaks a secret.
- **Explain & interview themes:** venvs vs. installing globally, mutable default arguments, the error family tree, why `print` debugging doesn't scale, what `__init__.py` does.
- **Move on when:** you can build a small, multi-file Python project from scratch, with logging and config, without being told the file layout.

Document 02 — APIs, HTTP, JSON & Backend Basics

- **Depends on:** Doc01.
- **Topics:** HTTP verbs/status codes, REST basics, `requests`, headers/logins (API keys, bearer tokens), timeouts, retries with backoff, checking JSON, rate limits.
- **Learn:** what actually happens on the wire when your Python code calls an outside API — and every way that call can fail.
- **Practice:** call a public API with `requests`, handle a 429, handle a timeout, handle bad JSON.
- **Build:** a small `http_client.py` wrapper with a timeout and retry-with-backoff, reused later by every project.
- **Break & debug:** a fake timeout, a 401, a 500, a cut-off JSON response — figure it out from the error alone, before I tell you what broke.
- **Explain & interview themes:** repeating a call safely (idempotency), retry storms, exponential backoff + jitter, why you never blindly retry a 400.
- **Move on when:** given just a stack trace, you can tell whether a failure is your bug or the server's.

Document 03 — LLM Basics

- **Depends on:** Doc02.
- **Topics:** tokens, context windows, temperature/top_p, prompting basics, system vs. user vs. assistant roles, cost (input/output token pricing), speed vs. quality trade-offs, making things up as a normal, expected trait (not a bug).
- **Learn:** what an LLM is actually doing when it answers — enough that "it made something up" stops feeling like a random mystery, and becomes an expected failure type you design around.
- **Practice:** guess token counts by hand, then check with a tokenizer tool; compare answers across different temperature settings on the same prompt.
- **Build:** nothing yet — this document is about ideas, feeding directly into Doc04.
- **Break & debug:** a prompt that goes over the context limit; a temperature setting that makes the answer different every time, when a test expects the same answer.
- **Explain & interview themes:** why LLMs have no memory between calls, what a "context window" really limits, why token cost isn't just "how much I asked for."
- **Move on when:** you can guess, before running it, roughly how a prompt will behave and roughly what it'll cost.

Document 04 — OpenAI API → Project 1

- **Depends on:** Docs 01-03.
- **Topics:** the `openai` Python library, Chat Completions API, message lists, streaming, structured output (Pydantic shapes), the basic shape of tool/function calling, error types

(`RateLimitError` , `AuthenticationError` , `APITimeoutError`), the Responses API (overview).

- **Learn:** the raw API, before any library touches it — every library you learn next is a wrapper around this.
- **Practice:** one call → a multi-turn conversation → a streamed reply → a checked, structured reply.
- **Build — Project 1 (Beginner LLM App):** a terminal chatbot with conversation memory, proper `.env` -based key handling, and one structured-output feature (like returning a checked JSON object at least once).
- **Break & debug:** a bad API key, going over the context limit, a bad structured-output reply, a rate limit hit in the middle of a conversation.
- **Explain & interview themes:** Chat Completions vs. Responses API, streaming vs. non-streaming trade-offs, how structured output is actually enforced.
- **Move on when:** Project 1 runs start to finish, handles at least 3 of the failures above gracefully, and you can explain every line without help.

Document 05 — LangChain Basics

- **Depends on:** Doc04.
- **Topics:** why use a library on top of raw API calls, the `ChatOpenAI` wrapper, prompt templates, LCEL (connecting pieces with `|`), output parsers, the `langchain-core` vs. `langchain-openai` vs. `langchain` package split.
- **Learn:** what LangChain gives you over raw SDK calls, and — just as important — what it costs you (an extra layer, and more distance when debugging).
- **Practice:** rebuild a piece of Project 1's logic using LCEL instead of raw SDK calls; compare the two side by side.
- **Build:** a small, reusable "chain" module (prompt template → model → parser).
- **Break & debug:** a chain that quietly returns the wrong type, because a parser doesn't match what the prompt actually produces.
- **Explain & interview themes:** when LangChain is the right call vs. when the raw SDK is simpler and easier to debug, what LCEL actually connects when it runs.
- **Move on when:** you can decide, for a given task, LangChain vs. raw SDK — not just default to whichever you learned last.

Document 06 — Tools & Function Calling

- **Depends on:** Doc05.
- **Topics:** OpenAI's tool/function-calling shape, LangChain's `@tool` decorator, checking tool arguments with Pydantic, registering multiple tools, controlling tool choice, getting a tool's error back to the model.

- **Learn:** how a model "decides" to call a tool, and why that decision is a guess you need to check, not something to blindly trust.
- **Practice:** one tool → several tools with overlapping jobs (forces the model to choose) → a tool that can fail.
- **Build:** 2-3 real tools (like a calculator, a fake lookup, a real `requests`-based tool) registered on one model.
- **Break & debug:** the model calls the wrong tool, calls a tool with badly-formed arguments, or calls a tool twice when once was correct.
- **Explain & interview themes:** why tool descriptions are prompts too, how choosing a tool is different from "hoping the model picks right."
- **Move on when:** given a new tool description, you can register it correctly and guess when the model will and won't choose it.

Document 07 — AI Agents → Project 2

- **Depends on:** Doc06.
- **Topics:** the think→act→observe loop, LangChain's `create_agent` and its older predecessor `AgentExecutor` (worth knowing), the agent's step history, stopping conditions, hard step limits, common single-agent failures.
- **Learn:** an "agent" isn't magic — it's a loop around tool-calling with a stopping rule. Once that clicks, everything after this document is putting pieces together, not new magic.
- **Practice:** trace an agent's steps by hand before writing code that automates it.
- **Build — Project 2 (Tool-Using Agent):** a single agent with 3+ tools, checked inputs, and graceful handling of a tool failure without crashing the whole run.
- **Break & debug:** an agent stuck in a loop (no stopping rule), an agent that picks a tool but sends bad arguments, an agent that ignores a tool's error and makes up a result anyway.
- **Explain & interview themes:** the think→act→observe loop vs. plain function-calling, why step limits aren't optional in production, the cost of agent loops.
- **Move on when:** Project 2 survives all three problems above, without you having to guide it by hand.

Document 08 — RAG

- **Depends on:** Doc07.
- **Topics:** embeddings, ways to chunk text, vector stores (Chroma/FAISS locally), similarity search, search as a tool (not a separate pipeline bolted on), re-ranking (overview), the "lost in the middle" problem.
- **Learn:** RAG is just search results feeding into a prompt — nothing more mysterious than that, but every step (chunking, embedding, searching) has failure types that stack up.
- **Practice:** chunk a document 3 different ways and compare search quality; deliberately mismatch a question and its answer to see search fail.

- **Build:** a search module with one function, `retrieve(query) -> list[Document]` — this becomes a *tool* in Project 3, not its own separate app.
- **Break & debug:** bad chunk boundaries splitting an answer across two chunks, a search that confidently returns the wrong documents, a context limit going over because too many chunks got stuffed in.
- **Explain & interview themes:** chunking trade-offs, picking an embedding model, why RAG doesn't remove the "making things up" problem (it just changes what gets made up).
- **Move on when:** you can tell whether a bad RAG answer is a search problem or an answer-writing problem — that one skill is the whole point.

> **Required checkpoint before Document 09 — Async Python.** `async` / `await`, event loop basics, `asyncio.gather` for running things at the same time, sync vs. async client differences (`OpenAI` vs. `AsyncOpenAI`). Short and focused — taught in full only when you reach it, since LangGraph is the first place you actually need it.

Document 09 — LangGraph

- **Depends on:** Doc08 + the Async Python checkpoint.
- **Topics:** `StateGraph`, the state shape (`TypedDict` /Pydantic), nodes, edges, conditional edges, loops, checkpointers (saved state across runs), `interrupt()` for pausing for a human, streaming graph runs.
- **Learn:** LangGraph turns an agent from a hidden loop (Doc07's think→act→observe) into a clear, checkable graph — every state change is code you wrote and can test, not a black box.
- **Practice:** rebuild Project 2's agent as a graph before adding anything new — same behavior, a different, more checkable build.
- **Build:** the graph base for Project 3, connected to a checkpointer, with at least one conditional edge and one pause-for-a-human point.
- **Break & debug:** an endless loop from a missing/wrong conditional edge, state that doesn't survive a pause, a node that changes state the wrong way.
- **Explain & interview themes:** why clear state beats hidden agent memory, checkpointer trade-offs, when a graph is overkill vs. actually needed.
- **Move on when:** you can draw a graph's node/edge diagram from its code (or the other way), without running it.

Document 10 — Agent Workflows → Project 3

- **Depends on:** Doc09.
- **Topics:** connecting the Doc08 search tool as a graph node, combining tool-calling + branching + saved state into one working agent, streaming replies to whoever's using it, resuming after a human-pause.

- **Learn:** this document has no new basic ideas — it's about combining things. The skill is putting Docs 06-09 together into one working system, with no new ideas to hide behind.
- **Practice:** add the search tool to the graph piece by piece, testing after each connection instead of all at once.
- **Build — Project 3 (LangGraph App):** a LangGraph agent with state, branching, a RAG search tool, saved state, and at least one human-approval pause before a "risky" action.
- **Break & debug:** a race between the checkpointer and a mid-run pause; a routing decision that skips the search node when it shouldn't.
- **Explain & interview themes:** how this is different from Project 2, where saved state actually lives, what pausing for a human costs in delay/user experience.
- **Move on when:** Project 3 runs a full search → answer → (pause for approval) → resume → answer cycle, start to finish, in your own code.

Document 11 — Multi-Agent Systems → Project 4

- **Depends on:** Doc10.
- **Topics:** design patterns (single, sequential, supervisor, hierarchical, peer-to-peer, parallel, generator→critic), the `Command` tool for handoffs, shared vs. split state, agent-as-node vs. agent-as-subgraph, redundant-work and state-leaking failures.
- **Learn:** for every pattern above — what, why, when, when *not*, cost, speed, reliability, scaling, failure types. And the single most important judgment call in this whole curriculum: **one agent vs. many agents**, and how to defend that choice instead of just defaulting to "multi-agent sounds impressive."
- **Practice:** design (on paper, before code) the same task three ways — single agent, sequential agents, supervisor — and guess which one wins on cost/speed/reliability before building any of them.
- **Build — Project 4 (Multi-Agent System):** Supervisor → Research Agent → Analysis Agent → Writer Agent → Reviewer Agent, with routing, shared state, and a checker/reviewer step that can send work back for changes.
- **Break & debug:** the supervisor routes to the wrong agent, two agents redundantly call the same tool, a loop where reviewer and writer never agree, notes leaking into unfiltered shared state.
- **Explain & interview themes:** the trade-off table for every pattern, how you'd defend "why 4 agents and not 1" to a skeptical senior engineer.
- **Move on when:** Project 4 finishes a full task through all four agents, survives at least two of the failures above, and you can defend the design choice without being asked twice.

Document 12 — Production AI Engineering

- **Depends on:** Doc11.
- **Topics (by importance):**

- **Must-know:** FastAPI (routes, request/response models, error handling), keeping secrets safe, structured logging, retries/timeouts, basic rate limiting.
- **Important:** SQL/database basics (a schema for conversation/run history, an ORM or plain SQL — your call, we'll talk trade-offs), Docker (a Dockerfile for the app, why containers beat "it works on my machine"), caching (replies/embeddings).
- **Advanced:** login/permissions, basic CI/CD, queues for slow agent work, scaling across servers.
- **Optional:** advanced cost-saving (prompt caching, picking cheaper models for easier tasks), running in multiple regions.
- **Learn:** the gap between "a project that works when I run it" and "a system someone else can run, trust, and pay for."
- **Practice:** wrap Project 4 behind a FastAPI route step by step — one route, then saved data, then login.
- **Build:** a FastAPI service exposing Project 4 as an API, backed by a database for run/conversation history, packaged with Docker.
- **Break & debug:** a database connection leak under load, a secret accidentally committed to git, a container that works locally but fails from a missing environment variable, a timeout that snowballs into a retry storm.
- **Explain & interview themes:** how you'd scale this to 10x traffic, what you'd cut first under a cost limit, how secrets safely move from environment to container.
- **Move on when:** the Doc11 system is reachable over HTTP, saves state in a real database, and runs from a Dockerfile — not just `python main.py`.

Document 13 — Testing, Evaluation & Watching Your System

- **Depends on:** Doc12.
- **Topics:** unit/integration/API/tool/agent/workflow/end-to-end testing, regression testing for prompts, test sets, LLM-as-a-judge, testing for made-up answers, tool-choice accuracy, routing accuracy, logging/tracking runs (like LangSmith-style tracking), watching dashboards (overview).
- **Learn:** why testing a system that isn't always the same needs a different mindset than testing normal code — you're testing a range of behavior, not one fixed answer.
- **Practice:** write a normal, always-the-same unit test for a tool function, then a test for an LLM answer that can't be checked with `==`.
- **Build:** a small test suite for Project 4 — a fixed set of inputs with pass/fail or scored rules, run automatically.
- **Break & debug:** a test that fails randomly because it checks for exact LLM wording; a regression where a prompt change quietly made tool choices less accurate.
- **Explain & interview themes:** how you test something with no single right answer, LLM-as-judge pitfalls (judge bias, cost), what you'd track in production to catch a quiet quality drop.
- **Move on when:** you have a runnable test suite for Project 4 that would actually catch a regression if you (or I) sabotaged the prompts.

Document 14 — Debugging Lab (covers everything)

- **Depends on:** Docs 01-13.
- **Topics:** every failure type from your original request, practiced together instead of one at a time — Python errors, API failures, OpenAI-specific failures (going over context, bad structured output, tool-call failure, using too many tokens), LangChain failures (bad chains, parser failures), RAG failures, LangGraph failures (bad state, wrong edges, endless loops, checkpoint problems), multi-agent failures (wrong routing, agents disagreeing forever, repeated calls, notes leaking between agents, high cost/delay).
- **Method practiced:** Observe → Reproduce → Form a guess → Test it → Find the real cause → Fix → Test again → Prevent it happening again. No "just Google the error" shortcuts.
- **Practice:** I give you broken requirements + symptoms + logs + what should happen vs. what's actually happening. You figure it out before I confirm or correct you.
- **Build:** nothing new — this document stress-tests everything you've already built.
- **Explain & interview themes:** walk through a real debugging session, start to finish, out loud, as if I'm interviewing you.
- **Move on when:** you can work through at least 5 planted bugs across different layers (Python/API/LangGraph/multi-agent) using the 8-step method, on your own, without me nudging you.

Document 15 — Five Projects (reference index)

- Not new content — a standing page you come back to, linking each project brief to its document group (table in §2 above), plus the exact "plan before you code" process (§17 of your original request: Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks) done fresh at the start of every project.

Document 16 — System Design & Architecture

- **Depends on:** Doc14 (you need the failure-type vocabulary from it) and really Doc15 too.
- **Topics:** given a set of requirements (not a design), you design: agents, tools, state, database, APIs, RAG, routing, security, scaling, watching/monitoring — I review it like a senior architect would, I don't hand you the design.
- **Learn:** turning unclear business requirements into a design you can actually defend, and defending your trade-offs when questioned.
- **Practice:** 2-3 system-design prompts, getting bigger each time, written up by you before we discuss them.
- **Build:** design documents, not code — this is the "plan on paper before you type" skill.
- **Explain & interview themes:** a full system-design interview, played out.
- **Move on when:** your designs already account for the failure types from Doc14, without me having to ask.

Document 17 — Interview Preparation (covers everything)

- **Depends on:** every earlier document (it pulls them together).
- **Format for every question, always:** Question → your answer → my review → correction → the ideal answer, given at four levels of depth (30-second / normal / deep technical / senior-level).
- **What it covers:** basic/intermediate/advanced/scenario/debugging/system-design questions across every document, plus "why / why not X / how / what happens underneath / what happens if it fails / how would you scale it / cut its cost / debug it" as a standing question format for anything you've built.
- **Move on when:** you can answer a cold, scenario-based question about Project 4's design at a senior level, with no notes.

Document 18 — Final Production Multi-Agent Capstone → Project 5

- **Depends on:** everything.
- **Given to you (not designed for you):** business requirements, functional/non-functional requirements, limits, expected scale/users, security requirements, cost limits.
- **You design and build:** agents, tools, workflow, state, RAG, database, APIs, login, testing, watching/monitoring, deployment — reviewed by me like a senior architect at each stage, not corrected line-by-line unless you ask.
- **Move on when:** you can defend every design choice when pushed on it, without notes — then move to Doc19, the final document.

Document 19 — MLOps & LLMOps (Running What You Built)

- **Depends on:** Doc18 (Project 5 built and working).
- **Topics:** MLOps vs. LLMOps for prompt/agent-based systems, prompt/setting versions, CI/CD test gates (Doc13's suite as a release blocker), canary/blue-green/shadow releases, watching for slow decline (cost/speed/quality trends over time, not single-request failures), rollback triggers, backup plan if a provider has an outage.
- **Learn:** running a live system is a different skill from shipping it once — Doc12 got Project 5 live; this document is what safely changing it, on day 2 and beyond, actually needs.
- **Practice:** version two prompt versions and score them side by side; build a small release gate that blocks a worse version; act out a canary comparison; define and trigger a real rollback condition.
- **Build:** a versioned release-gate layer on top of Project 5 — prompt/setting versions, a test-gated `deploy()`, and a `rollback_to_previous()` that only ever goes back to a version that itself passed the gate.
- **Break & debug:** a release gate skipped by editing the "live" marker directly; a rollback that goes back to "the previous version" instead of "the previous version that *passed*"; a slow decline that only shows up as a trend, invisible to per-request alerts.

- **Explain & interview themes:** MLOps vs. LLMOps, why prompts need the same version discipline as code, canary vs. blue-green vs. shadow trade-offs, what a good rollback trigger looks like, a backup plan for a provider outage.
- **Move on when:** you have a working gate that actually blocks a worse version of Project 5 from reaching "live," and can explain what would make you trust an automatic rollback enough not to watch every release yourself. This is the real end of the curriculum — go back to §6 below to check every final ability is there.

---

5. Multi-agent design patterns (a preview — full detail in Doc11)

Pattern	One-line shape
Single agent	One loop, all tools
Sequential	A fixed order, agent → agent → agent
Supervisor	A router agent hands off to specialists, collects results
Hierarchical	Supervisors of supervisors
Peer-to-peer	Agents hand off directly to each other, no central router
Parallel	Independent agents run at the same time, results merged
Generator → Critic → Revision	One agent creates, another checks it, loop until it's good enough

The main question practiced throughout Doc11: **one agent vs. many** — multi-agent trades cost/speed/complexity for reliability/specialization. It's not a default choice.

---

6. Final capabilities (what "done" means)

By Document 19 you should be able to, on your own, without AI writing the code for you:

- Write production-quality Python for AI systems; work with HTTP APIs; use the OpenAI API (Chat Completions + Responses) correctly and safely.
- Build LangChain pieces and defend when to use them vs. raw SDK calls.
- Build tools, single agents, RAG pipelines, LangGraph workflows, and multi-agent systems.
- Choose and defend an agent design (§5) for a given problem.
- Debug failures across every layer using the 8-step method (Doc14), not search-and-guess.
- Write tests and evaluations for systems that aren't always the same.

- Build and ship a FastAPI service backed by a database, containerized (Docker: images, layers, keeping secrets safe, multi-stage builds), with logging/tracking/watching.
- Version prompts/settings, gate releases behind a test suite, and design a rollback plan — MLOps/LLMOps discipline for a live system, not just a one-time ship.
- Reason about security, cost, and scaling trade-offs, and explain them at interview depth.
- Design a system from unclear requirements and defend the design when questioned.

Status

Roadmap complete. Nothing has been taught yet — as you asked, I'm stopping here.

Say **START DOCUMENT 01** (or name any document/checkpoint above) when you're ready.